Orientación a Objetos II 2026

Explicación de práctica
Semana del 16 de marzo

FACULTAD DE INFORMATICA

UNIVERSIDAD NACIONAL DE LA PLATA

Staff de la práctica

JTPs

Agustín Ortu
Germán Ruiz
Juan Cruz Gardey
Leonardo Loza Bonora

Ayudantes

Diego De Sogos
Cristian Cerro
Nero Brindesi
Luciana Tanevitch
Felipe Dioguardi
Ignacio Traberg
Guadalupe Martin
Catalina Zuppa Moro
Martin Tau
Giuliana Maltempo

Adscriptos/colaboradores

Gomez Torres Rafael Genaro
Nathaniel Frutos
Camila Tarrio

Joaquin Tartaruga
Bautista Clua
Lautaro Gubia
Bianca Defelipe
Matheo Lamiral
Augusto Boschiazzo

Consultas prácticas

- **Inscripción a las comisiones**
  - Todos tienen que estar inscriptos en alguna comisión.
  - Si tienen problemas de superposición laboral o con alguna otra materia, escribir a los JTPs a **oo2@lifia.info.unlp.edu.ar**

- **Consultas**
  - Se deben respetar los turnos asignados para la práctica para que podamos garantizar un espacio semanal de consulta para cada estudiante.
  - El primer cuadernillo de ejercicios ya está publicado. Estar atentos a la fecha de actualización por si se modifica su contenido.

Actividades prácticas

Explicación de práctica: 2 turnos

- **Lunes 18 a 19 hs**; Aula 5
- **Martes 8:30 a 9:30 hs;** Aula 10A

Consultas de práctica: 7 horarios

- Lunes 19:00 a 20:30 hs., aula 5 (aula sin pcs)
- Martes 10:00 a 11:30 hs., aula 1-3 (aula sin pcs)
- Martes 17:30 a 19:00 hs, aula 1-1
- Martes 19:00 a 20:30 hs., aula 1-1
- Miércoles 17:30 a 19:00 hs, aula 1-4 (aula sin pcs)
- Miércoles 19:00 a 20:30 hs., aula 2 (aula sin pcs) + aula 1-3 (aula sin pcs)
- Jueves 10:00 a 11:30 hs., sala de PC.

Para aprobar la cursada

Para **aprobar la cursada** se requiere la **aprobación** de un **examen presencial**

- Primera fecha: Sábado 6 de junio, 13:00hs
- 1er Recuperatorio: Sábado 27 de junio, 13:00hs
- 2do Recuperatorio: Sábado 11 de julio, 13:00hs

Aulas: 4, 5, 9, 10A y 11

El parcial **se aprueba por temas**

- Refactoring
- Patrones de diseño
- Frameworks

Java

Deben tener instalado

- Alguna versión de JDK (puede ser Open JDK - https://openjdk.java.net/ versión 11+)

- Un entorno de desarrollo, por ejemplo Eclipse (https://www.eclipse.org/downloads/)

Se puede usar cualquier otro IDE que prefieran

Ver “Material Introductorio (extraído de Objetos 1)”

Bibliografía recomendada

- Design Patterns. Elements of Reusable Objects Oriented Software. Gamma, Helm, Johnson, Vlissides, Addison-Wesley.
- Refactoring: Improving the Design of Existing Code. Fowler, Martin. Addison-Wesley. 1ra edición
- Refactoring to Patterns. Joshua Kerievsky. Addison Wesley.
- Implementing Application Frameworks: Object-Oriented Frameworks at Work. Fayad, Schmidt, Johnson (eds).

Guías y material de referencia

Trabajar con proyectos Maven en OO2

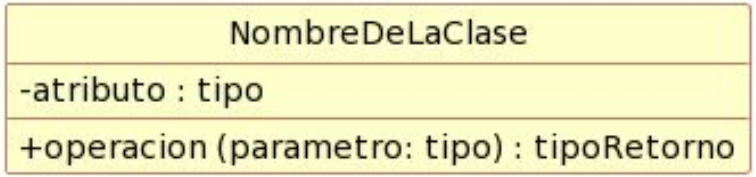
Diagramas UML - Resumen

¿Cómo preguntar en el foro?

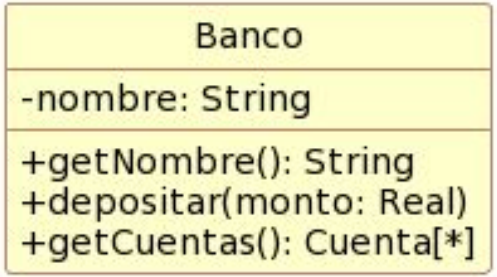
Cheatsheet Streams

Pueden encontrar el siguiente material en el curso en Moodle

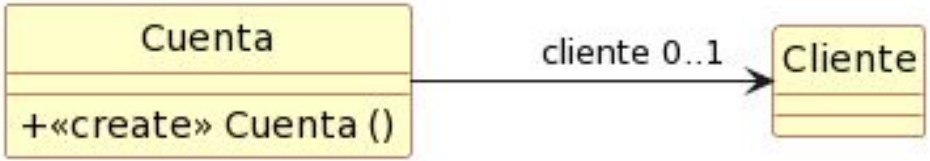
UML - Repaso de lo visto en OO1

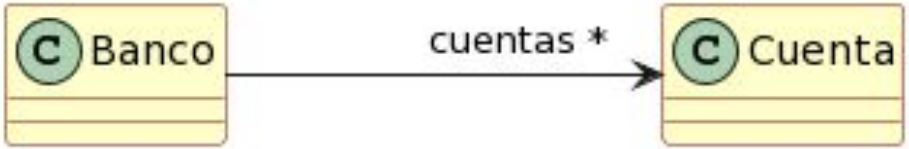


Ejemplo



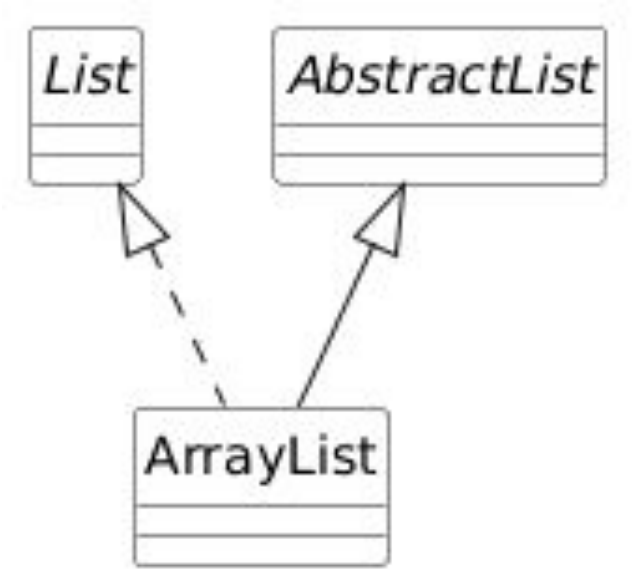
Asociaciones entre clases





Ver [“Diagramas UML - Resumen”] en el curso de Moodle

UML - PlantUML



- No es obligatorio usar esta herramienta, es una sugerencia
- https://plantuml.com/ -> opción online server
- https://plantuml.com/es/class-diagram -> documentación sintaxis
- Link diagrama del slide

```
@startuml
interface List {}
abstract class AbstractList {}
class ArrayList extends
AbstractList implements List {}
@enduml
```

Sobre los ejercicios prácticos

Diseño - Implementación

Asegurarse de aplicar adecuadamente las heurísticas y buenas prácticas de diseño vistas en Objetos 1

Tests

Deben cubrir las clases del modelo cubriendo el protocolo y considerando casos interesantes, según lo visto en OO1*.
Usar JUnit 5.

* se pueden obviar getters y setters**
** verificar que los getters y setters están justificados.

Ejemplo de un test en Java

```
1  public class PersonaTest {
2      private Persona james, guido;
3
4      @BeforeEach
5      void setUp() throws Exception {
6          james = new Persona();
7          guido = new Persona();
8          james.setApellido("Glosing");
9          james.setNombre("James");
10         guido.setApellido("van Rossum");
11         guido.setNombre("Guido");
12
13     }
14
15     @Test
16     public void testNombreCompleto() {
17         assertEquals("Glosing, James", james.getNombreCompleto());
18         assertEquals("van Rossum, Guido", guido.getNombreCompleto()
19     }
20 }
```

Necesario para que lo reconozca el ambiente

Creamos los objetos que vamos a probar

Verificamos los resultados esperados con “assertEquals”

Ejercicios de la práctica

- Los ejercicios se organizan en cuadernillos.
- Se actualizarán periódicamente (aconsejamos no imprimir!)
- Habrá 4 cuadernillos. Por el momento, sólo el de Repaso está disponible
  - Repaso
  - Refactoring
  - Patrones
  - Frameworks

Trabajos Prácticos

Cuadernillo de Repaso

Sobre las explicaciones de práctica…

Sobre las explicaciones…

- En las explicaciones de práctica, sólo presentamos los ejercicios que deben resolverse para ir al día con la materia.

- Tengan presente que los ejercicios siguientes podrían resolverlos igual. Sin embargo, cuando reciba la teoría correspondiente, encontrará una mejor técnica para hacerlo.

- En este contexto es importante **conversar y validar las soluciones** con los ayudantes en las consultas
  - Aunque funcione, hay que validarlo para ver si hay una alternativa mejor.
  - Aunque aplique patrones, hay que validar que se hayan aplicado correctamente.

Actividades de la semana

Cuadernillo de repaso

- Ejercicio 1 - Red social
- Ejercicio 2 - Piedra papel o tijera

Esta semana deben trabajar y resolver estos ejercicios

Cuadernillo de repaso

- **Ejercicio 1 - Red Social**
- Ejercicio 2: Piedra Papel o Tijera

Ejercicio 1: Red social

Se quiere programar en objetos una versión simplificada de una red social parecida a Twitter. Este servicio debe permitir a los usuarios registrados postear y leer mensajes de hasta 280 caracteres. Ud. debe modelar e implementar parte del sistema donde nos interesa que quede claro lo siguiente:

- Cada usuario conoce todos los Tweets que hizo.
- Un tweet puede ser re-tweet de otro, y este tweet debe conocer a su tweet de origen.
- Twitter debe conocer a todos los usuarios del sistema.
- Los tweets de un usuario se deben eliminar cuando el usuario es eliminado. No existen tweets no referenciados por un usuario.
- Los usuarios se identifican por su screenName.
- No se pueden agregar dos usuarios con el mismo screenName.
- Los tweets deben tener un texto de 1 carácter como mínimo y 280 caracteres como máximo.
- Un re-tweet no tiene texto adicional.

Tareas

Su tarea es diseñar y programar en Java lo que sea necesario para ofrecer la funcionalidad antes descrita. Se espera que entregue los siguientes productos.

1. Diagrama de clases UML.
2. Implementación en Java de la funcionalidad requerida.
3. Implementar los tests (JUnit) que considere necesarios.

Cuadernillo de repaso

- Ejercicio 1 - Red Social
- **Ejercicio 2: Piedra Papel o Tijera**

Ejercicio 2: Piedra Papel o Tijera

Se quiere programar en objetos una versión del juego Piedra Papel o Tijera. En este juego dos jugadores eligen entre tres opciones: piedra, papel o tijera. La piedra aplasta la tijera, la tijera corta el papel, y el papel envuelve la piedra. Los jugadores eligen una opción y se determina un ganador según las reglas:

-

	Piedra	**Papel**	**Tijera**
Piedra	Empate	Papel	Piedra
Papel	Papel	Empate	Tijera
Tijera	Piedra	Tijera	Empate

→ Pensar una solución con polimorfismo y delegación.